2026-2027 ACADEMIC YEAR

WEB SERVICES

LAB PRACTICE 1 GUIDE AND EXPERIMENTS

INSTRUCTIONS TO STUDENTS FOR PREPARING A LAB REPORT

This Lab Manual is prepared to help the students with their practical understanding and development of skills, and is used as a base reference during the lab/practical classes. Students have to submit Lab experiment report of this lab practice at 4:30 PM /11:30AM AND at 1:30 PM/8:30AM for previous lab into corresponding next lab, and the lab report to be submitted should include at least the following topics listed clearly in documentation (comments in your program code) at the beginning of each file:

1. **GROUP NAME**
2. **GROUP MEMBERS NAME AND ID**
3. **Title of the program**
4. **Output (compilation)**

All codes must compile. Code that does not compile will not be graded.

Before installing and configuring the tools for web service lab practice for implementing java web services, let's introduce JWSDP as below.

Introduction

The Java Web Services Developer Pack (JWSDP) is a collection of tools and libraries designed to make the development of Web services in Java as painless as possible. First introduced in January 2002 as an Early Access (EA) release, the JWSDP brings together XML parsers, SOAP support, service containers, and build tools that can be used to create and consume widely distributed services based on XML protocols.

At the time of this writing, the current release is EA2, which contains the following components:

■ The JAXP XML processing libraries

■ The JAXM XML messaging libraries

■ The JAX-RPC XML Remote Procedure Call libraries

■ The JAXR libraries for accessing XML registries

■ The Java Secure Sockets (JSSE) library

■ JSP Standard Tag Libraries

■ The Apache/Jakarta Ant build tool

■ The Apache/Jakarta Tomcat servlet container

■ A simple UDDI registry (WSDP Registry tool)

■ The Web Application Deployment tool

You probably noticed that parts of the JWSDP were not written by Sun. In fact, the JWSDP is mostly a collection of existing products; the JWSDP acts to package these tools in a form convenient to download and install. Also, with the JWSDP you know you have versions of the tools that work together, saving a lot of frustration when trying to get some code working.

The focus is on the libraries and tools that come with the JWSDP, and using these tools for real-world applications. After practicing this tool, you should be ready to begin writing and using Web services and XML documents in Java. Let's first look at the following components of the JWDSP:

Ant and Tomcat

Like JSTL, Ant comes from Apache Jakarta. Ant isn't a Java library—rather, it's a build tool. It's included with the JWSDP simply because it makes writing and bundling Java applications easy to do. Ant comes with built-in tasks for creating WAR (Web Application aRchive), which are the default packing for Java Web Services. Ant is an open-source project, whose sources can be found on www.apache.org

Similarly, Apache Jakarta's Tomcat is a reference implementation of the JSP and servlet standards. As such, it allows for the development hosting of servlet-based Web services, such as JAXM supports. Unlike Ant, you likely will not use Tomcat for production deployments; it does not support failover, reliable message delivery, EJBs, or a host of other production features. However, it is free, fast, and lightweight to install; you can easily run Tomcat as a test server on your development machine. Like Ant, Tomcat is open-source and available at www.jakarta.apache.org/ant.

Along with Ant and Tomcat, JWSDP includes the WSDP (Web Services Developer Package) Registry server. This is a simple UDDI server, again for development and testing use. Unlike Ant and Tomcat, WSDP is a Sun project and is not open-source.

Let's discuss Ant, Tomcat, and WSDP Registry as tools used in JWSDP.

- JWSDP UDDI Registry
- Tomcat
- Ant

Introduction

Included in the JWSDP are a set of tools that can be used in the development or deployment of Web services. These tools can be employed to support a development server, and can even be used for some production releases. The tools include a UDDI registry (Java WSDP registry server), a servlet/JSP container (Tomcat), and a build tool (Ant).

The Java WSDP registry server is least likely to be used in production environments—at least in its current form. It has limitations in the scope of UDDI supported, its capability to provide highly available services, and in its performance. These will likely be addressed in future releases, so it will be interesting to see if it eventually evolves into a production-level tool.

Tomcat is a much more capable tool. Behind IBM's WebSphere and BEA's WebLogic, it represents one of the most accomplished and up-to-date servlet engines. It may even be suitable for small production environments. This depends on your overall willingness to assume risk, of course. Tomcat is generally supported by open-source volunteers, and does not provide high-availability features.

If used in development, **Ant** is the tool you would likely continue to use as you moved toward production. As a build tool, the issues of high availability are moot. Ant is as capable a tool for Web services as any other make utility—perhaps even more so. Keep in mind that it's a very different tool than standard make utilities, and does take some getting used to.

JWSDP UDDI Registry

The Universal Description Discovery and Integration Protocol (UDDI) is an infrastructure that allows Web services to register, or become publicly known by, a registry. The registry used by Java WSDP is known as the UDDI Registry. Once these Web services are known to the registry, they are then made available to any client request. The UDDI Registry contents are XML-based and are accessed in a directory tree-style.

In order to deploy your Web services, you will need to set up the UDDI Registry, enabling you to add your Web services to the registry. To begin, install and configure the Java Web Services Developer Pack, containing the Registry Server, as below:

GUIDE:

- **INSTALLING AND CONFIGURING LAB TOOLS**

You need to install and configure three tools for web service lab practice. The tools will be for implementing java web services. These tools, which will be provided to you from the instructor through your representatives, are

i. Java(TM) Web Services Developer Pack 2.0

It is advised to be installed in the following location: C:\Sun\jwsdp-2.0 with the following features:

JAX-WS Version 2.0 EA (Java API for XML Web Services)

Fast Infoset Version 1.0.1

Sun Java Streaming XML Parser Version 1.0 EA

XML Digital Signature Version 1.0.1

XML and Web Services Security Version 2.0 EA2

JAXB Version 2.0 EA (Java Architecture for XML Binding)

JAXP Version 1.3.1_01 (Java API for XML Processing)

JAXR Version 1.0.8_01 EA (Java API for XML Registries)

JAX-RPC Version 1.1.3_01 EA (Java API for XML-based RPC)

SAAJ Version 1.3 EA (SOAP with Attachments API for Java)

for a total size: 89.7 MB

ii. install and configure JDK 1.5.x
iii. install and configure tomcat 5.

Please remember that this software is in zip file and it is advised to be extracted in the following location: C:\.

Configuring

You need to add the /bin directory of the JWSDP to the PATH variable in your environment. This will then allow you to run any of the JWSDP executables from any directory. For Microsoft Windows users, modify your PATH variable through the Control Panel | System | Advanced | Environmental Variables section. Once this section is accessed, modify the existing PATH setting by appending the following directory path: C:\Sun\jwsdp-2.0\jwsdp-shared\bin. Check your Microsoft Windows manual on instructions for configuring environmental variables for your version of Windows.

Similarly, add the /bin directory of the:

JDK 1.5.x to the PATH variable in your environment. For Microsoft Windows users, modify your PATH variable through the Control Panel | System | Advanced | Environmental Variables section. Once this section is accessed, modify the existing PATH setting by appending the following directory path: C:\Program Files\Java\jdk1.5.0_22, and C:\Program Files\Java\jdk1.5.0_22\bin

tomcat 5. to the PATH variable in your environment. For Microsoft Windows users, modify your PATH variable through the Control Panel | System | Advanced | Environmental Variables section.

Once this section is accessed, modify the existing PATH setting by appending the following directory path: C:\tomcat50-jwsdp, C:\tomcat50-jwsdp\apache-ant, and C:\tomcat50-jwsdp\bin.

Activating

Once the JWSDP is installed and configured, the JAXR Registry Server can be easily launched in Windows by executing the startup script for the Tomcat server, which is located under the C:\tomcat50-jwsdp\bin directory. Unix and Linux users execute the startup.sh script, whereas the Microsoft Windows users execute the startup.bat batch file. Likewise, the Server can be shut down in the same manner using the shutdown script for both platforms.

cd C:\tomcat50-jwsdp\bin.

C:\tomcat50-jwsdp\bin.> startup.bat

There will be a text output to be displayed on the CMD screen. And, a dialog box will appear.

Experiment 1

a. What is the text output to be displayed on the CMD screen?
b. Screenshot and show the dialog box that will appear.

Selecting as the Registry

The Registry Server has a browser-based tool used to administer the server. Prior to activating the Registry Browser, make sure Tomcat is running. See the Tomcat section for instructions on starting your Tomcat server. You can activate the Registry Browser by a method that s to execute the jaxr-browser.bat (or jaxr-browser.sh) program at a DOS prompt. This program is located under C:\Sun\jwsdp-2.0\jaxr\bin directory:

cd C:\Sun\jwsdp-2.0\jaxr\bin

C:\Sun\jwsdp-2.0\jaxr\bin>jaxr-browser

A dialog box will appear.

Experiment 2

Screenshot and show the dialog box that will appear.

Select a registry using the dialog box drop-down list. You will be able to access the last registry in the list, provided that Tomcat is currently running. The last registry in the list provides UDDI Registry samples. Administration of any Registry Server requires some level of authentication. If you are interested in adding Web services to these registries, you must go to one (or both) of these sites and become a registered user with a username and password: http://uddi.microsoft.com or http://www-3.ibm.com/services.

Experiment 3:

Go to one (or both) of the above sites and become a registered user with a username and password and screenshot and show a listing of the test registries.

Experiment 4:

Explain in detail and write a report of 3 pages about the JAXR (UDDI) Registry Browser. Explain how you can select the Submissions tab using this browser menu, to add, modify, edit, or delete Web services to within your Registry Server. Explain about organization, primary contact information, classifications and service binding.

Experiment 5:

Go to this page: localhost:8080, and write a report of half a page of what you saw.

Experiment 6:

Go to this page: http://localhost:8080/admin/frameset.jsp, and write a report of half a page of what you saw.

Experiment 7:

Go to this page: http://localhost:8080/manager/html, and write a report of half a page of what you saw.

Experiment 8:

Go to this page: http://localhost:8080/tomcat-docs/, and write a report of half a page of what you saw.

Experiment 9:

Go to this page: http://localhost:8080/tomcat-docs/appdev/index.html, and write a report of half a page of what you saw.

Experiment 10:

Go to this page: http://localhost:8080/tomcat-docs/appdev/sample/build.xml, and write a report of half a page of what you saw.

Experiment 11:

Go to this page: http://localhost:8080/tomcat-docs/security-manager-howto.html, and write a report of half a page of what you saw.

Experiment 12:

Access the server.xml file that is located under the C:\tomcat50-jwsdp\conf\ directory. And explain the structure and use of this file in a report of one page.

Experiment 13:

Explain the webapps directory, context paths, directory called WEB-INF

and Web ARchive (WAR) files for the Tomcat Web server in a report of two pages.

Experiment 14:

Access the web.xml configuration file that is located under the C:\tomcat50-jwsdp\webapps\webdav\WEB-INF directory, and explain the structure and use of this file in a report of one page.

Experiment 15:

Explain the Ant that is an open-source build tool, structure of an Ant input file such as a project definition, target directories, and any tasks that need to be performed during the build, how to invoke the Ant script from the command-line in a report of four pages.

Experiment 16:

A **simple Apache Ant build file** that compiles a servlet and deploys it to **Tomcat 5**.
Below is a **complete, runnable example** that assumes:

- Tomcat 5 is installed locally.
- Your servlet source is in src/ and package structure matches.
- You have a WEB-INF/web.xml in web/WEB-INF/.
- Tomcat's servlet-api.jar is available in TOMCAT_HOME/common/lib/.

Crete build.xml as below:

```
<?xml version="1.0" encoding="UTF-8"?>

<project name="SimpleServletBuild" default="deploy" basedir=".">

  <!-- ===================== -->

  <!-- Project configuration -->

  <!-- ===================== -->

  <property name="src.dir" value="src"/>

  <property name="build.dir" value="build"/>

  <property name="web.dir" value="web"/>

  <property name="tomcat.home" value="C:/tomcat50-jwsdp"/>

  <property name="deploy.dir" value="${tomcat.home}/webapps/YourApp"/>
```

```
  <!-- Classpath includes servlet API -->
  <path id="compile.classpath">
    <pathelement location="${tomcat.home}/common/lib/servlet-api.jar"/>
  </path>
  <!-- ==================== -->
  <!-- Clean build directory -->
  <!-- ==================== -->
  <target name="clean">
    <delete dir="${build.dir}"/>
  </target>
  <!-- ==================== -->
  <!-- Compile servlet source -->
  <!-- ==================== -->
  <target name="compile" depends="clean">
    <mkdir dir="${build.dir}"/>
    <javac srcdir="${src.dir}" destdir="${build.dir}" includeantruntime="false"
classpathref="compile.classpath"/>
  </target>
  <!-- ==================== -->
  <!-- Prepare WAR structure -->
  <!-- ==================== -->
  <target name="prepare" depends="compile">
    <mkdir dir="${build.dir}/WEB-INF/classes"/>
    <copy todir="${build.dir}/WEB-INF/classes">
      <fileset dir="${build.dir}"/>
    </copy>
    <copy todir="${build.dir}/WEB-INF">
```

```
        <fileset dir="${web.dir}/WEB-INF"/>
      </copy>
    </target>
    <!-- ==================== -->
    <!-- Deploy to Tomcat -->
    <!-- ==================== -->
    <target name="deploy" depends="prepare">
      <mkdir dir="${deploy.dir}"/>
      <copy todir="${deploy.dir}">
        <fileset dir="${build.dir}"/>
      </copy>
      <echo message="Servlet deployed to ${deploy.dir}. Restart Tomcat to apply changes."/>
    </target>
</project>
```

Directory Structure Example

```
project/
  ├── build.xml
  ├── src/
  |    └── com/example/MyServlet.java
  ├── web/
  |    └── WEB-INF/
  |         └── web.xml
  └── /path/to/tomcat5/
       └── common/lib/servlet-api.jar
```

Example Servlet (src/com/example/MyServlet.java)

```
package com.example;
import java.io.IOException;
```

```
import javax.servlet.ServletException;
import javax.servlet.http.*;
public class MyServlet extends HttpServlet {
   @Override
   protected void doGet(HttpServletRequest req, HttpServletResponse resp)
         throws ServletException, IOException {
      resp.setContentType("text/plain");
      resp.getWriter().println("Hello from MyServlet!");
   }
}
```

How to Run

1. Install Tomcat 5 and set tomcat.home in build.xml.
2. Place your servlet source in src/ and web.xml in web/WEB-INF/.
3. Project base dir set to: C:\tomcat50-jwsdp\webapps\YourApp
4. Run: ON CMD

```
C:\tomcat50-jwsdp\webapps\YourApp>ant deploy
```

Show the screenshot of the outcome as a result of running the Ant file.